

Marketing Video Creator — Source Code Documentation

Technical documentation for the web client (`client/web`). Intended for a developer who is taking over, extending, or reselling the codebase.

1. Overview

Marketing Video Creator is a **100% client-side** single-page web app. There is no backend: uploads, editing state, and video processing all happen in the browser. Three features are implemented:

- 1. **Split** — cut one video into multiple files.
- 2. **Insert** — insert clips into a base video at marked positions.
- 3. **Decor** — burn text overlays into a video.

Video processing is done with **ffmpeg compiled to WebAssembly**. Projects are persisted in **IndexedDB** (including the raw video blobs), so work survives a page reload. Exports are written to disk with the **File System Access API** (Chrome/Edge), falling back to normal downloads elsewhere.

Tech stack

Concern	Choice
Framework	React 18
Language	TypeScript (strict)
Build tool	Vite 5

State management	Zustand
Local persistence	IndexedDB via <code>idb-keyval</code>
Video processing	<code>@ffmpeg/ffmpeg</code> + <code>@ffmpeg/core</code> (wasm)
Styling	Hand-written CSS (single stylesheet)

No CSS framework, no component library, and no network calls — the dependency surface is intentionally small.

2. Getting started

```
cd client/web
npm install      # also vendors the ffmpeg core into public
npm run dev      # http://localhost:5173
npm run build    # type-check (tsc -b) + production build
npm run preview  # serve the production build locally
```

Requirements: Node 18+ (developed on Node 22). No other services required.

The `public/ffmpeg/` folder is generated from `node_modules/@ffmpeg/core` by `scripts/copy-core.mjs`, which runs automatically before `dev` and `build` (`predev` / `prebuild` npm hooks). It is gitignored because it is a large, regenerated binary.

3. Project structure

```
client/web/
├─ index.html           # Vite entry HTML
├─ vite.config.ts       # Vite config (React plugin)
├─ package.json         # scripts + deps
```

```

├─ scripts/copy-core.mjs      # copies ffmpeg ESM core i
├─ public/ffmpeg/           # (generated) ffmpeg-core.
└─ src/
    ├─ main.tsx              # React root
    ├─ App.tsx                # top bar + routes between
    ├─ styles.css             # all styling + CSS design
    ├─ types.ts               # domain model (Project, C
    ├─ store.ts               # Zustand store: state + a
    ├─ db.ts                  # IndexedDB persistence (p
    ├─ vite-env.d.ts          # Vite client types
    └─ lib/
        ├─ ffmpeg.ts          # ffmpeg load + exportSpli
        ├─ save.ts             # File System Access API s
        ├─ filmstrip.ts        # useFilmstrip hook – time
        ├─ textRender.ts       # canvas text → PNG + shar
        └─ format.ts           # time/byte formatting + p
    └─ components/
        ├─ Dashboard.tsx      # feature cards + recent-p
        ├─ Editor.tsx          # editor shell, upload sta
        ├─ DecorWorkspace.tsx # Decor editor: draggable
        ├─ Timeline.tsx        # filmstrip timeline with
        ├─ ExportModal.tsx     # export options + progres
        ├─ Toast.tsx           # tiny toast store + compo
        └─ Icons.tsx           # inline SVG icon set

```

Approx. 3,000 lines of source. Largest files: `Editor.tsx`,
`DecorWorkspace.tsx`, `ffmpeg.ts`.

4. Data model (`src/types.ts`)

```

type FeatureType = "split" | "insert" | "decor";
type OutputFormat = "mp4" | "mov";
type Quality = "original" | "high" | "medium";

interface VideoMeta { name; mimeType; size; duration; w
interface Cut { id; time }

```

```

interface InsertPoint { id; time; clip?: VideoMeta }
interface TextOverlay {
  id; text;
  x; y; // center position as 0..1 fraction
  fontSize; // in VIDEO pixels (not preview pixels)
  fontFamily; bold; italic; color;
  background; // "none" | "dark" | "light" readability
  start; end; // visibility window in seconds
}

interface Project {
  id; name; type; createdAt; updatedAt;
  video?: VideoMeta; // base video metadata
  cuts: Cut[]; // used by Split
  inserts: InsertPoint[]; // used by Insert
  overlays: TextOverlay[]; // used by Decor
  format: OutputFormat; quality: Quality;
}

```

One `Project` shape covers all three features; the unused arrays stay empty. `createProject(type)` returns a blank project with a `crypto.randomUUID()` `id`.

Coordinate conventions worth knowing:

- Overlay `x/y` are fractions (0..1) so they are resolution-independent.
- Overlay `fontSize` is stored in **video pixels**; the preview scales it by `renderedHeight / videoHeight` so preview and export match exactly.

5. State management (`src/store.ts`)

A single Zustand store holds everything:

- `route("dashboard" | "editor"), projects, current` (open project), `videoUrl` (object URL for the base video), `busy`.

- **Navigation:** `init`, `newProject`, `openProject`, `closeEditor`, `removeProject`.
- **Editing:** `patchCurrent` (the core mutator — merges a patch, bumps `updatedAt`, and persists), `setBaseVideo`, plus feature-specific actions: `addCut/removeCut`, `addInsert/removeInsert/setInsertClip`, `addOverlay/updateOverlay/removeOverlay`.

Every edit funnels through `patchCurrent`, which writes to IndexedDB on each change, so there is no explicit "save" button — projects autosave. `openProject` backfills any arrays missing from older saved projects (`overlays` `??= []`, etc.) for forward compatibility.

Object URLs for the base video are created on load and revoked on close/replace to avoid memory leaks.

6. Persistence (`src/db.ts`)

Two IndexedDB object stores via `idb-keyval`:

- **mvc-projects** → project records keyed by id (small JSON).
- **mvc-blobs** → raw media keyed by `video:<projectId>` and `clip:<insertId>` (large Blobs).

Separating metadata from blobs keeps the project list cheap to read.

API: `listProjects`, `getProject`, `saveProject`, `deleteProject` (also deletes the project's video + clip blobs), and blob helpers `saveVideoBlob/getVideoBlob`, `saveClipBlob/getClipBlob/deleteClipBlob`.

7. Video processing (`src/lib/ffmpeg.ts`)

The heart of the app. `ffmpeg` is lazy-loaded once via `getFFmpeg()` and cached.

Why the ESM core, self-hosted: the ffmpeg worker runs as a *module* worker and loads the core with `import (coreURL) .default`, which requires the **ESM** build of `@ffmpeg/core` (the UMD build has no default export and fails). The core is served **same-origin** from `public/ffmpeg/` to avoid CORS/CDN issues and to work offline. `CORE_BASE` points at `window.location.origin + BASE_URL`.

Helpers:

- `run(ff, args)` — executes one command, checks the exit code, and throws with the tail of the captured ffmpeg log on failure.
- `getRecentLogs()` — exposes recent log lines so the UI can show real errors.
- A single `progress` listener delegates to a mutable `progressCb` so each export step can map ffmpeg's 0..1 progress into an overall ratio.

Three export functions, each returning `ExportedFile { name, blob }`:

- **exportSplit** — for each segment between cut boundaries, re-encodes with `-ss START -i input -t DURATION -c:v libx264 -crf ... -c:a aac`. Re-encoding (not stream copy) is deliberate: it makes cuts **frame-accurate** so a clip never repeats the tail of the previous one. Returns one file per segment.
- **exportInsert** — builds an ordered timeline of base slices and inserted clips, normalizes each piece to the base resolution as MPEG-TS (`base = scale=W:H; clips = scale=... increase, crop=W:H → fill & center-crop`), then concatenates with the concat demuxer. This lets clips of any resolution/codecs join cleanly.
- **exportDecor** — takes pre-rendered transparent PNGs (one per overlay) and composites them with a chained `overlay` filtergraph, time-gated per overlay via `enable=between(t, start, end)`.

`CONTAINER` maps format → extension/MIME; `CRF` maps quality → x264 CRF (original 18, high 20, medium 24).

8. Text rendering (`src/lib/textRender.ts`)

Text overlays are **not** drawn with `ffmpeg drawtext` (which needs fonts compiled into the wasm core). Instead, `renderOverlayPng(overlay, W, H)` rasterizes each overlay onto a full-frame transparent `<canvas>` and returns a PNG blob, which `exportDecor` overlays onto the video.

The payoff: the **preview and export use the same rendering math**, so what you see is what you get. Shared constants (`SCRIM_PAD_X/Y`, `SCRIM_RADIUS`), `fontShorthand()`, and `scrimColor()` are used by both the canvas renderer and the live preview in `DecorWorkspace.tsx`, keeping them pixel-consistent.

9. Saving to disk (`src/lib/save.ts`)

Two-phase to respect browser security rules:

- `pickSink(count, suggestedName)` — opens the save/folder picker **during the click gesture** (before the long `ffmpeg` step, because the picker requires transient user activation that expires). Returns a `Sink` describing a chosen directory, a chosen file, or download mode; `null` if the user cancelled.
- `writeSink(sink, files)` — after export, writes the blobs to the chosen location, or triggers downloads in fallback mode.

`showDirectoryPicker` is used when there are multiple output files (`Split`); `showSaveFilePicker` for a single file. Firefox/Safari lack these APIs, so they fall back to downloads.

10. UI components

- **App.tsx** — renders the top bar and switches between `Dashboard` and `Editor` based on route. Calls `init()` once on mount.
- **Dashboard.tsx** — three feature cards (`newProject(type)`) and a recent-projects list from IndexedDB with open/delete.
- **Editor.tsx** — the editor shell (header with editable title, autosave indicator, export button). Shows an upload dropzone until a video is loaded, then renders the Split/Insert workspace, or delegates to `DecorWorkspace` for Decor. Contains the shared preview + transport + side panels for Split/Insert.
- **DecorWorkspace.tsx** — the Decor editor. Measures the letterboxed video rectangle with a `ResizeObserver`, renders draggable text on top, and provides the text style panel (font, size, color, bold/italic, background scrim, timing with "now" buttons). Overlays follow their time window on playback and show a dimmed "ghost" when the selected one is scrubbed outside its window while paused.
- **Timeline.tsx** — the filmstrip timeline: frame thumbnails, a scrubbable playhead, and feature-specific markers (cut lines + clip labels for Split, insert markers for Insert, none for Decor).
- **ExportModal.tsx** — format/quality options, output preview, progress bar, and the orchestration that calls `pickSink` → the right `export*` function → `writeSink`, surfacing ffmpeg errors and logs on failure.
- **Toast.tsx** — a minimal Zustand-based toast. **Icons.tsx** — inline SVGs.

`filmstrip.ts` exposes `useFilmstrip(videoUrl, duration, count)`, which seeks a hidden `<video>` and captures evenly-spaced frames to a canvas (best-effort; degrades to placeholders). `format.ts` has `formatTime`, `toFfmpegTime`, `formatBytes`, `relativeTime`, and `probeVideo` (reads duration/width/height).

11. Styling & theming (`src/styles.css`)

All styling lives in one stylesheet driven by CSS custom properties in `:root` — colors, borders, radius, shadow. It ships a **light theme** with a

per-feature accent color: Split = amber (`--split`), Insert = green (`--insert`), Decor = violet (`--decor`). Re-theming (including a dark variant) is mostly a matter of changing these tokens. Layout is responsive; the side panel collapses on narrow screens.

12. Browser support & limitations

- **Recommended: Chrome or Edge** — required for the "choose save location" dialog (File System Access API). Other browsers download to the default folder instead.
 - ffmpeg here is **single-threaded** (no `SharedArrayBuffer` / cross-origin isolation needed). It is reliable and works offline, but large/long videos are slow and memory-heavy. For heavy workloads, the upgrade path is the multi-threaded core (needs COOP/COEP headers) or a native-ffmpeg backend.
 - HEVC/H.265 and other exotic source codecs may not decode in the wasm build.
-

13. Extending the app

To add a new feature/tool:

1. Add the type to `FeatureType` and any new fields to `Project` in `types.ts`.
2. Add store actions in `store.ts` (mutate via `patchCurrent`).
3. Add an `export*` function in `lib/ffmpeg.ts`.
4. Add a dashboard card in `Dashboard.tsx`, a `workspace/panel` UI, and an `ExportModal.tsx` branch.
5. Add an accent token in `styles.css` if desired.

The existing three features are good templates — Split and Insert share the generic workspace in `Editor.tsx`; Decor shows how to build a dedicated one.

14. Third-party dependencies & licensing

Package	Purpose	License
react / react-dom	UI framework	MIT
zustand	State management	MIT
idb-keyval	IndexedDB wrapper	MIT
@ffmpeg/ffmpeg, @ffmpeg/util	wasm ffmpeg driver	MIT
@ffmpeg/core	ffmpeg wasm build	MIT (wrapper)
vite, @vitejs/plugin-react	build tooling	MIT

Important for resale: `@ffmpeg/core` bundles FFmpeg compiled to WebAssembly. FFmpeg itself is licensed under **LGPL/GPL** depending on the build (the standard `@ffmpeg/core` build is LGPL-2.1; some builds enable GPL components). If you redistribute this source commercially, review the FFmpeg and `@ffmpeg/core` licenses and include the required attributions/notices. All first-party application code in `src/` is yours to license as you wish.